

Book Recommendation System

Quick Reference Guide

Overview

This is a simplified, practical implementation of key LangChain concepts using a book recommendation system instead of the complex fashion product assistant.

Setup

Before Running:

```
pip install langchain langchain-core langchain-openai langchain-google-genai langchain-huggingface
python-dotenv pandas pydantic
```

Model Agnostic Setup

KEY CONCEPT: Switch models without changing your code!

```
llm = init_chat_model(
    model='openai:gpt-4o-mini', # Change this line
    temperature=0.7,
    max_tokens=500
)
```

Supported Models:

Provider	Model String
OpenAI	"openai:gpt-4o-mini" or "openai:gpt-4"
Google Gemini	"google_genai:gemini-2.5-flash-lite"
HuggingFace	"huggingface:meta-llama/Llama-2-7b-chat"

Key Concepts

- 1. Messages:** Build conversations with clear roles (System, Human, AI). Use SystemMessage, HumanMessage, AIMessage.
- 2. Document Loader:** Load data from CSV, PDF, etc. using CSVLoader, PyPDFLoader, etc.
- 3. Document Injection:** Inject external data into prompts using ChatPromptTemplate.
- 4. Output Parser:** Get structured JSON/objects using StrOutputParser, PydanticOutputParser.
- 5. LCEL Chaining:** Connect components with | operator: prompt | llm | parser
- 6. Control Flow (RunnableBranch):** Route to different chains based on conditions.
- 7. Generate-Refine:** Multi-step pipeline: generate → transform → refine

8. Message History: Multi-turn conversations with memory using `RunnableWithMessageHistory`.

9. Serialization: Save/load chains as JSON using `dumps()` and `loads()`.

Common Patterns

Pattern	Code Example
Simple Recommendation	<code>chain = prompt llm StrOutputParser()</code>
Structured Output	<code>chain = prompt llm PydanticOutputParser()</code>
Conditional Routing	<code>RunnableBranch((condition1, chain1), (condition2, chain2))</code>
Multi-Step	<code>chain1 (lambda x: {"data": x}) chain2</code>
With Memory	<code>RunnableWithMessageHistory(chain, get_history, ...)</code>

Debugging Tips

- 1. **Check prompt:** Add print before LLM
- 2. **Inspect chain:** `chain.invoke(...)` shows each step
- 3. **Use LangSmith:** Set `LANGCHAIN_TRACING_V2` for production

Files in This Package

- 1. **book_recommendation.ipynb** - Full Jupyter notebook with sample data
- 2. **book_recommendation_kaggle.ipynb** - Notebook with real Kaggle dataset
- 3. **book_recommendation.py** - Standalone Python script
- 4. **DATASETS_GUIDE.pdf** - Guide to public datasets
- 5. **QUICK_REFERENCE.pdf** - This file